

Lec 10

3rd Aug, 2026

Prefix and Postfix Expression Evaluation — Comprehensive Notes

1. Background: Why Not Just Use Infix?

Infix expressions (e.g. $A + B * C$) are natural for humans but hard for machines because evaluation requires knowledge of **operator precedence** and **associativity**, plus **parentheses** to override precedence. This forces multi-pass parsing or complex lookahead logic.

Prefix and **postfix** notations remove ambiguity entirely — there is exactly one way to parse them, no precedence rules or parentheses are ever needed. This is why compilers and calculators internally convert expressions to postfix/prefix before evaluation.

Notation	Operator Position	Example (for $A + B$)
Infix	Between operands	$A + B$
Prefix (Polish)	Before operands	$+ A B$
Postfix (Reverse Polish, RPN)	After operands	$A B +$

Full Example

Infix: $(A + B) * (C - D)$

- Prefix: $* + A B - C D$
- Postfix: $A B + C D - *$

2. Key Properties

1. **No parentheses needed** — the position of operators fixes the structure.
2. **No precedence/associativity rules needed** during evaluation — order is already encoded.
3. Both can be evaluated in **single left-to-right (postfix)** or **single right-to-left (prefix)** pass using a **stack**.
4. Every valid expression corresponds to a unique **expression tree** (binary tree where leaves are operands and internal nodes are operators):

- Postfix = **Post-order traversal** of the expression tree.
- Prefix = **Pre-order traversal** of the expression tree.
- Infix = **In-order traversal** of the expression tree.



Pre-order (prefix): $* + A B - C D$ Post-order (postfix): $A B + C D - *$ In-order (infix, no parens): $A + B * C - D$ (ambiguous — this is WHY infix needs parens/precedence)

3. Postfix (Reverse Polish Notation) Evaluation

Core Idea

Scan left to right. Operands are pushed onto a stack. When an operator is encountered, pop the required number of operands (2 for binary operators), apply the operator, and push the result back.

Algorithm: Evaluate Postfix Expression

ALGORITHM EvaluatePostfix(expression):

Step 1: Initialize an empty stack S

Step 2: for each token in expression (left to right):

Step 2a: if token is an operand (number/variable value):
push token onto S

Step 2b: else if token is an operator (+, -, *, /, ^, etc.):
operand2 = pop from S # NOTE: pop order matters!
operand1 = pop from S
result = apply operator(operand1, operand2)
push result onto S

Step 3: return pop from S # final result

Why operand1, operand2 order matters

Postfix $A\ B\ -$ means $A - B$, NOT $B - A$. Since B is pushed after A, B is popped FIRST. So: $\text{operand2} = \text{pop}()$ (this is B), $\text{operand1} = \text{pop}()$ (this is A), compute $\text{operand1} - \text{operand2} = A - B$. This ordering is critical for non-commutative operators ($-$, $/$, $^$).

Worked Example

Evaluate postfix: $6\ 2\ 3\ +\ -\ 3\ 8\ 2\ /\ +\ *$

Token	Action	Stack (bottom→top)
6	push	[6]
2	push	[6, 2]
3	push	[6, 2, 3]
+	pop 3,2 → $2+3=5$, push	[6, 5]
-	pop 5,6 → $6-5=1$, push	[1]
3	push	[1, 3]
8	push	[1, 3, 8]
2	push	[1, 3, 8, 2]
/	pop 2,8 → $8/2=4$, push	[1, 3, 4]
+	pop 4,3 → $3+4=7$, push	[1, 7]
*	pop 7,1 → $1*7=7$, push	[7]

Result = 7

Complexity

- Time: $O(n)$ — each token processed once, each push/pop $O(1)$.
- Space: $O(n)$ — stack in worst case (all operands before any operator).

4. Prefix (Polish Notation) Evaluation

Core Idea

Scan **right to left**. Operands pushed onto stack. When operator found, pop two operands — first popped is treated as the LEFT operand, second popped as RIGHT operand (opposite convention vs postfix because we're scanning backwards).

Algorithm: Evaluate Prefix Expression

ALGORITHM EvaluatePrefix(expression):

Step 1: Initialize an empty stack S

Step 2: for each token in expression (RIGHT to LEFT):

Step 2a: if token is an operand:

push token onto S

Step 2b: else if token is an operator:

operand1 = pop from S # left operand

operand2 = pop from S # right operand

result = apply operator(operand1, operand2)

push result onto S

Step 3: return pop from S # final result

Worked Example

Evaluate prefix: $* + 2 3 - 4 1$ (equivalent infix: $(2 + 3) * (4 - 1)$)

Reverse scan order: $1 4 - 3 2 + *$

Token	Action	Stack (bottom→top)
1	push	[1]
4	push	[1, 4]
-	pop 4(op1), 1(op2) → 4-1=3, push	[3]
3	push	[3, 3]
2	push	[3, 3, 2]
+	pop 2(op1), 3(op2) → 2+3=5, push	[3, 5]
*	pop 5(op1), 3(op2) → 5*3=15, push	[15]

Result = 15

Complexity

- Time: **$O(n)$**
- Space: **$O(n)$**

5. Infix → Postfix Conversion (Shunting Yard Algorithm)

This is the most commonly tested algorithm (Dijkstra's Shunting Yard).

Precedence & Associativity Table (typical)

Operator	Precedence	Associativity
<code>^</code> (exponent)	3 (highest)	Right to Left
<code>*</code> , <code>/</code> , <code>%</code>	2	Left to Right
<code>+</code> , <code>-</code>	1 (lowest)	Left to Right
<code>(</code> <code>)</code>	—	special handling

Algorithm

ALGORITHM InfixToPostfix(expression):

Step 1: Initialize empty stack S (for operators)

Step 2: Initialize empty output list/queue OUTPUT

Step 3: for each token in expression (left to right):

Step 3a: if token is an operand:
append token to OUTPUT

Step 3b: else if token == '(':
push token onto S

Step 3c: else if token == ')':
while S is not empty AND top of S != '':
pop from S, append to OUTPUT
pop the '(' from S and discard it
(error if no '(' found – mismatched parens)

Step 3d: else if token is an operator (say op1):
while S is not empty AND top of S is an operator (say
op2) AND
 (precedence(op2) > precedence(op1) OR
 (precedence(op2) == precedence(op1) AND op1 is
left-associative))
 AND top of S != '':
pop op2 from S, append to OUTPUT
push op1 onto S

Step 4: while S is not empty:
pop from S, append to OUTPUT # (error if '(' remains –

mismatched parens)

Step 5: return OUTPUT

Worked Example

Convert infix $A + B * C - D$ to postfix.

Token	Stack (top→...)	Output
A	—	A
+	+	A
B	+	A B
*	+ (since > +)	A B
C	* +	A B C
-	- (pop * then + since - has $\leq$ precedence)	A B C * +
D	-	A B C * + D
end	pop all	A B C * + D -

Postfix: $A B C * + D -$

Verify: $A + (B * C) - D \checkmark$

Complexity

- Time: $O(n)$
- Space: $O(n)$ (stack + output)

6. Infix → Prefix Conversion

Algorithm (3-step trick using postfix logic)

ALGORITHM InfixToPrefix(expression):

1. Reverse the infix expression string.
 - While reversing, swap every '(' with ')' and every ')' with '('.
2. Convert this modified expression to "postfix" using the standard Infix→Postfix algorithm, with ONE change:
 - When precedences are EQUAL, use RIGHT-to-LEFT preference (i.e., treat as if right-associative) instead of left-to-right,

because we are effectively processing in reverse.
3. Reverse the resulting postfix expression to get the prefix expression.

Worked Example

Infix: $A + B * C - D$

Step 1 — Reverse + swap brackets: $D - C * B + A$

Step 2 — Convert to postfix (apply infix→postfix rules, equal precedence → right-to-left grouping): $D C B * - A +$ (walking through: $D \rightarrow \text{out}$; $- \rightarrow \text{push}$; $C \rightarrow \text{out}$; $* \rightarrow \text{push}$ (higher prec); $B \rightarrow \text{out}$; then next $-$ has equal prec to stack top $-$ but since we want right-to-left tie-breaking... let's verify carefully below)

Let's trace precisely:

Token	Stack	Output
D	—	D
-	-	D
C	-	D C
*	* -	D C
B	* -	D C B
-	(pop : $D C B$) then compare - vs - : equal prec, right-to-left $\Rightarrow$ do NOT pop, push	- -
A	- -	D C B * A
end	pop all	D C B * A - -

Hmm, discrepancy — let's redo carefully with correct rule application.

Step 2 (careful trace) on $D - C * B + A$:

Token	Action	Stack	Output
D	operand	—	D
-	push (stack empty)	-	D
C	operand	-	D C
*	$\text{prec}(*) > \text{prec}(-)$ on stack $\rightarrow$ just push	* -	D C
B	operand	* -	D C B

Token	Action	Stack	Output
+	prec(+) vs top(): + is lower $\Rightarrow$ pop $\rightarrow$ output; prec(+) vs new top(-): equal, right-to-left $\Rightarrow$ don't pop, push +	+ -	D C B *
A	operand	+ -	D C B * A
end	pop + then -	—	D C B * A + -

Step 3 — Reverse the postfix output: D C B * A + - reversed $\rightarrow$ - + A * B C D

Prefix: - + A * B C D

Verify:

- * B C = B*C
- + A (B*C) = A + B*C
- - (A+B*C) D = A + B*C - D $\checkmark$ matches original infix.

Complexity

- Time: $O(n)$
- Space: $O(n)$

7. Postfix $\leftrightarrow$ Prefix Direct Conversions

Postfix $\rightarrow$ Prefix

ALGORITHM PostfixToPrefix(expression):

Step 1: Initialize empty stack S

Step 2: for each token in expression (left to right):

Step 2a: if token is operand:

push token onto S

Step 2b: else if token is operator:

operand2 = pop from S

operand1 = pop from S

new_string = token + operand1 + operand2 # concatenate,

operator first

push new_string onto S

Step 3: return pop from S

Example: Postfix `A B + C D - * →`

- A,B pushed → push "+AB"
- C,D pushed → push "-CD"
- pop "-CD","+AB" → push "*+AB-CD"

Prefix: `* + A B - C D`

Prefix → Postfix

ALGORITHM PrefixToPostfix(expression):

Step 1: Initialize empty stack S

Step 2: for each token in expression (RIGHT to LEFT):

Step 2a: if token is operand:

push token onto S

Step 2b: else if token is operator:

operand1 = pop from S

operand2 = pop from S

new_string = operand1 + operand2 + token # concatenate,
operator last

push new_string onto S

Step 3: return pop from S

Example: Prefix `* + A B - C D` (scan right→left: D, C, -, B, A, +, *)

- D push, C push
- '-' → pop C(op1), D(op2)? — careful: right-to-left scan means first-popped is "left" operand
Actually apply same op1/op2 convention as prefix evaluation: op1=pop (first popped),
op2=pop(second popped), result = op1+op2+token = "C"+"D"+"-" = "CD-"... Let's be
precise: scanning right to left `D C - B A + *`:
 - D → push
 - C → push [stack: D, C]
 - '-' → op1 = pop() = C, op2 = pop() = D → new = op1+op2+token = "C"+"D"+"-" = "CD-"
→ push
 - B → push
 - A → push [stack: "CD-", B, A]
 - '+' → op1 = pop() = A, op2 = pop() = B → new = "A"+"B"+"+" = "AB+" → push [stack:
"CD-", "AB+"]
 - '*' → op1 = pop() = "AB+", op2 = pop() = "CD-" → new = "AB+"+"CD-"+"*" = "AB+CD-"

Postfix: A B + C D - * ✓ matches Section 1 example.

8. Handling Multi-Digit Numbers / Multi-Character Tokens

Real implementations tokenize the input first (split into a list of tokens) rather than scanning character-by-character, otherwise `12 + 3` could be misread as `1 , 2 , + , 3`. Use whitespace-delimited tokens or a proper lexer that reads full numbers before treating a character as an operator.

9. Common Edge Cases & Errors to Handle

- 1. **Division by zero** — check before applying `/`.
 - 2. **Mismatched parentheses** in infix conversion — if `)` found with no matching `(`, or stack has leftover `(` at end → invalid expression.
 - 3. **Insufficient operands** — if a stack pop fails (empty stack) when an operator is processed → malformed expression.
 - 4. **Leftover stack items** — after processing all tokens, if more than one item remains on the stack → malformed expression (too many operands, not enough operators).
 - 5. **Unary operators / unary minus** — need special-case handling (often via a distinct symbol, e.g. `~` or `u-`, with high precedence, right-associative).
 - 6. **Right-associativity of $^$** — e.g. $2 ^ 3 ^ 2 = 2 ^ (3 ^ 2) = 512$, not $(2^3)^2 = 64$. This must be respected in precedence comparisons during infix conversion.
-

10. Complexity Summary Table

Operation	Time	Space
Evaluate Postfix	O(n)	O(n)
Evaluate Prefix	O(n)	O(n)
Infix → Postfix	O(n)	O(n)
Infix → Prefix	O(n)	O(n)
Postfix → Prefix	O(n)	O(n)
Prefix → Postfix	O(n)	O(n)

n = number of tokens in the expression. All are linear because each token is pushed/popped from the stack a constant number of times (amortized $O(1)$ per token).

11. Quick Reference Cheat Sheet

Task	Scan Direction	Pop Order Convention
Evaluate Postfix	Left $\rightarrow$ Right	op2=pop() , op1=pop() , compute op1 OP op2
Evaluate Prefix	Right $\rightarrow$ Left	op1=pop() , op2=pop() , compute op1 OP op2
Postfix $\rightarrow$ Prefix	Left $\rightarrow$ Right	build string: OP + op1 + op2
Prefix $\rightarrow$ Postfix	Right $\rightarrow$ Left	build string: op1 + op2 + OP
Infix $\rightarrow$ Postfix	Left $\rightarrow$ Right	Shunting Yard (operator stack, output queue)
Infix $\rightarrow$ Prefix	—	reverse+swap parens $\rightarrow$ postfix (tie-break R $\rightarrow$ L) $\rightarrow$ reverse result

12. Sample Code (Python) — Postfix Evaluator

```
def evaluate_postfix(expr):
    stack = []
    ops = {'+', '-', '*', '/', '^'}
    for token in expr.split():
        if token not in ops:
            stack.append(float(token))
        else:
            b = stack.pop()
            a = stack.pop()
            if token == '+': stack.append(a + b)
            elif token == '-': stack.append(a - b)
            elif token == '*': stack.append(a * b)
            elif token == '/': stack.append(a / b)
            elif token == '^': stack.append(a ** b)
    return stack.pop()
```

```
# Example
print(evaluate_postfix("6 2 3 + - 3 8 2 / + *")) # 7.0
```

13. Sample Code (Python) — Prefix Evaluator

```
def evaluate_prefix(expr):
    stack = []
    ops = {'+', '-', '*', '/', '^'}
    for token in reversed(expr.split()):
        if token not in ops:
            stack.append(float(token))
        else:
            a = stack.pop()
            b = stack.pop()
            if token == '+': stack.append(a + b)
            elif token == '-': stack.append(a - b)
            elif token == '*': stack.append(a * b)
            elif token == '/': stack.append(a / b)
            elif token == '^': stack.append(a ** b)
    return stack.pop()
```

```
# Example
print(evaluate_prefix("* + 2 3 - 4 1")) # 15.0
```

14. Sample Code (Python) — Infix to Postfix (Shunting Yard)

```
def infix_to_postfix(expr):
    prec = {'+':1, '-':1, '*':2, '/':2, '%':2, '^':3}
    right_assoc = {'^'}
    output = []
    stack = []
    for token in expr.split():
        if token not in prec and token not in ('(', ')'):
            output.append(token)
        elif token == '(':
            stack.append(token)
        elif token == ')':
            while stack and stack[-1] != '(':
                output.append(stack.pop())
            stack.pop() # discard '('
        else: # operator
            while (stack and stack[-1] != '(' and
                   (prec[stack[-1]] > prec[token] or
                    (prec[stack[-1]] == prec[token] and token not in right_assoc))):
                output.append(stack.pop())
            stack.append(token)
```

```

right_assoc))) :
    output.append(stack.pop())
    stack.append(token)
while stack:
    output.append(stack.pop())
return ' '.join(output)

# Example
print(infix_to_postfix("A + B * C - D")) # A B C * + D -

```

15. Parenthesis Checker (Balanced Parentheses)

Before evaluating or converting an expression, it's common practice to first verify that all brackets are balanced — every opening bracket has a matching, correctly-ordered closing bracket. This is a classic stack application.

Algorithm

```

ALGORITHM IsBalanced(expression):
    Step 1: Initialize an empty stack S

    Step 2: for each character c in expression (left to right):
        Step 2a: if c is an opening bracket ( '(', '{', '[' ):
            push c onto S

        Step 2b: else if c is a closing bracket ( ')', '}', ']' ):
            if S is empty:
                return FALSE # closing bracket with no matching
open
                top = pop from S
                if top does not match c (wrong type/pair):
                    return FALSE

    Step 3: if S is empty:
        return TRUE # all opened brackets were closed properly
    else:
        return FALSE # unmatched opening bracket(s) remain

```

Worked Example

Check { [A + (B * C)] - D }

Char	Action	Stack
{	push	{
[	push	{ [
(	push	{ [(
)	pop (, matches	{ [
]	pop [, matches	{
}	pop { , matches	(empty)

Stack empty at end → **Balanced** ✓

Counter-example: (A + B]) → when] is encountered, top of stack is (, which does not match] → **Not Balanced**

Complexity

- Time: **O(n)**
- Space: **O(n)** worst case (all opening brackets, e.g. (((()

Sample Code (C++)

```
#include <iostream>
#include <stack>
#include <string>
using namespace std;

bool isBalanced(const string &expr) {
    stack<char> s;

    for (char c : expr) {
        // Step 2a: opening bracket -> push
        if (c == '(' || c == '{' || c == '[') {
            s.push(c);
        }
        // Step 2b: closing bracket -> check and pop
        else if (c == ')' || c == '}' || c == ']') {
            if (s.empty()) {
                return false; // closing bracket with nothing to match
            }
            char top = s.top();
            s.pop();
        }
    }
    return s.empty();
}
```

```

        if (c == ')') && top != '(') return false;
        if (c == '}') && top != '{') return false;
        if (c == ']') && top != '[') return false;
    }
    // any other character (operand/operator) is ignored
}

// Step 3: balanced only if stack is empty at the end
return s.empty();
}

int main() {
    string tests[] = {
        "{[A+(B*C)]-D}",
        "(A+B]",
        "((A+B)",
        "A+B*C"
    };

    for (const string &t : tests) {
        cout << t << " -> " << (isBalanced(t) ? "Balanced" : "Not Balanced")
<< endl;
    }

    return 0;
}

```

Expected Output:

```

{[A+(B*C)]-D} -> Balanced
(A+B]) -> Not Balanced
((A+B) -> Not Balanced
A+B*C -> Balanced

```

16. Summary

- Postfix/Prefix eliminate the need for precedence rules and parentheses during evaluation — a single stack-based linear scan suffices.
- Postfix = post-order traversal, Prefix = pre-order traversal of the expression tree.
- Evaluation direction differs: **postfix scans left→right**, **prefix scans right→left**.
- Operand pop order is critical for non-commutative operators (− , /).

- Infix→Postfix uses the Shunting Yard algorithm with an explicit operator stack and precedence/associativity comparisons.
- Infix→Prefix is typically done by reversing the string (with bracket swap), applying a modified infix→postfix, then reversing the result.
- All these algorithms run in **$O(n)$ time and $O(n)$ space**.